

Using sensors in the simulation environment

Before attempting this tutorial, it is recommended to familiarise yourself with the EGH446 simulation environment. Before you start,

- Download the simulation environment from Canvas.
- Read **Getting Started with EGH446 Simulation Environment** included in the main folder.

Objectives:

- Familiarise with the use of Lidars within the simulation environment.
- Use the visualisation and obstacle detection blocks available in the mobile robotics simulation toolbox

1. Examples

1.1. Using a Lidar

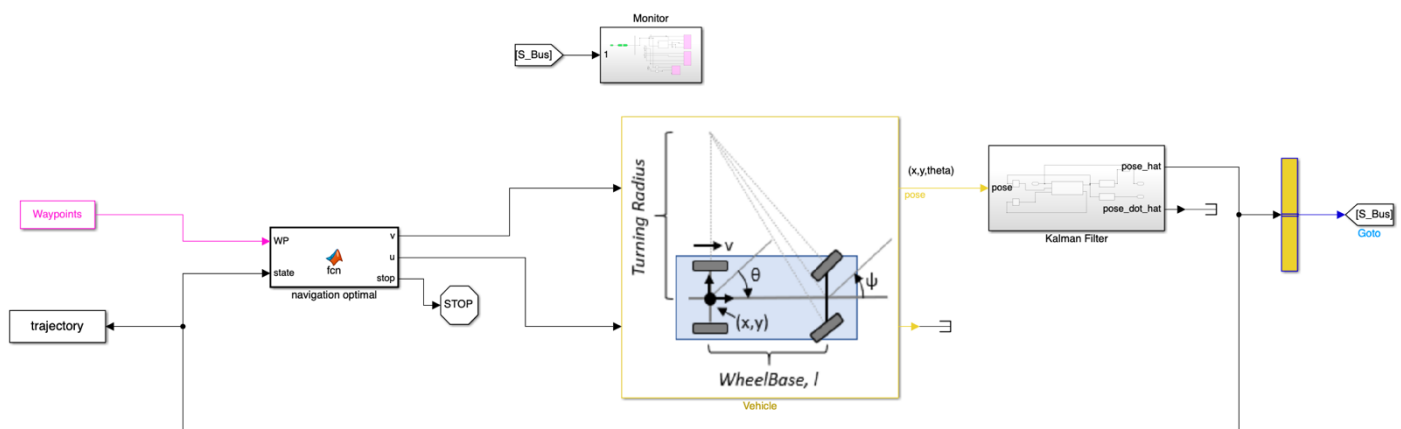
Before attempting this task, it is recommended to read the working principle of the model (mrsDocLidarSensor.mlx) in the directory:

toolboxes->MRTB->doc

Start the ground robot Simulink model developed by running

```
startupGround
```

Let's assume we have developed a basic guidance to direct the vehicle to a waypoint. This should display a Simulink model as follows:



I. Load the map and obstacles

A map can be added to the environment by running

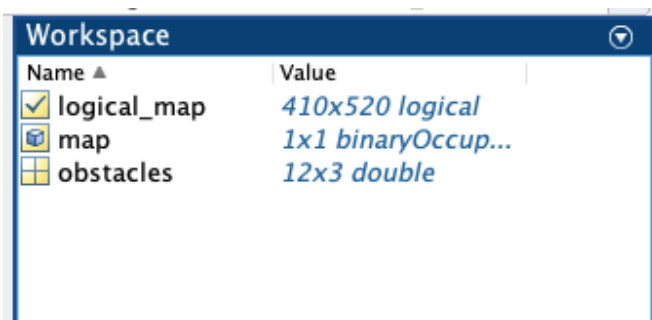
```
load('complexMap_air_ground.mat');
```

A variable **map** of type *binaryOccupancyMap* and a variable **logical_map** an array should appear in your workspace.

Obstacles can be loaded by

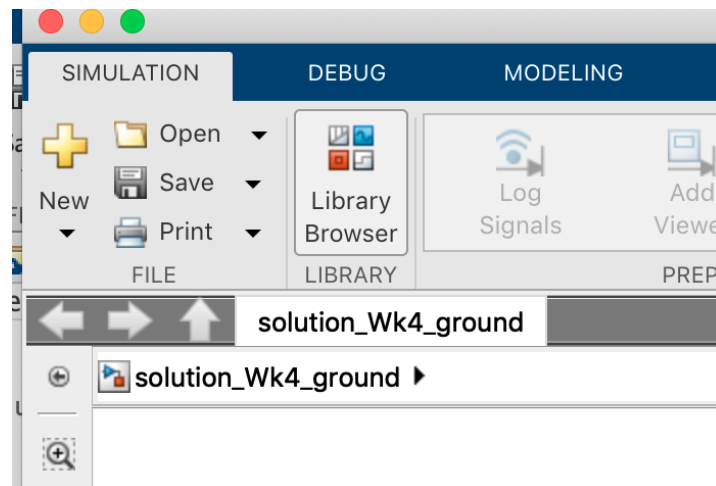
```
load('obstacles_air_ground.mat');
```

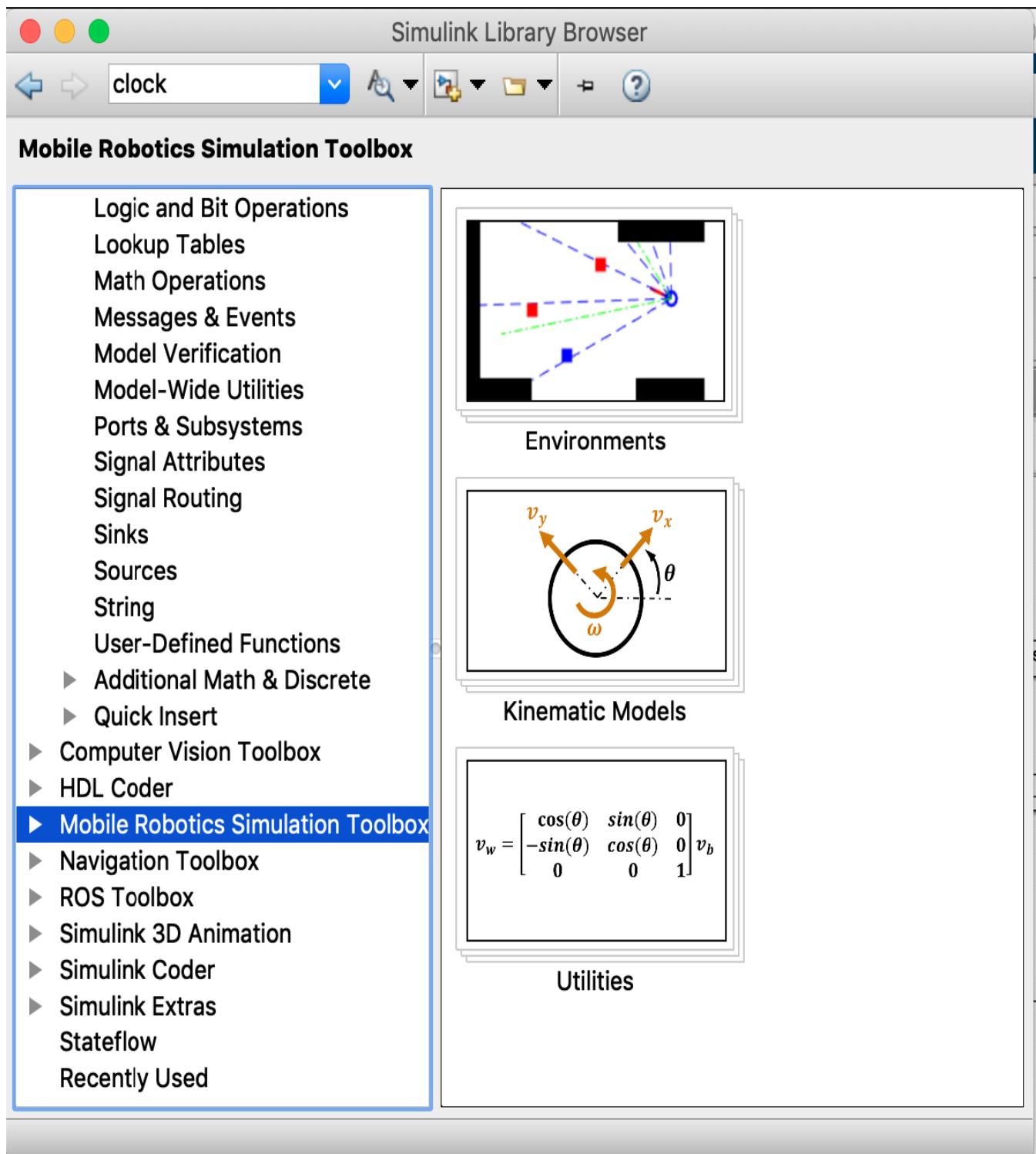
A variable **obstacles** which is an array of 12x3 should appear in your workspace

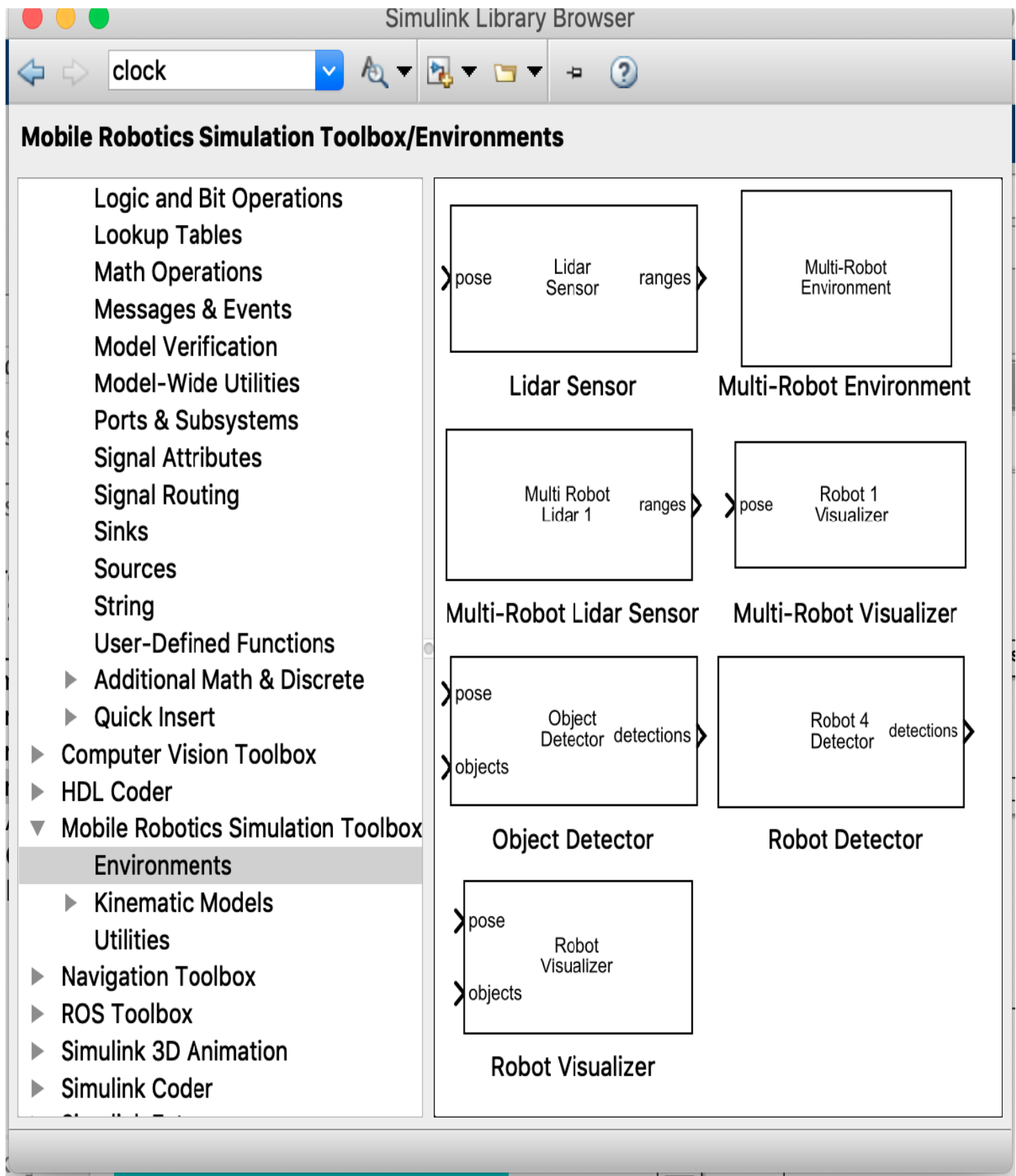


II. Add a Lidar

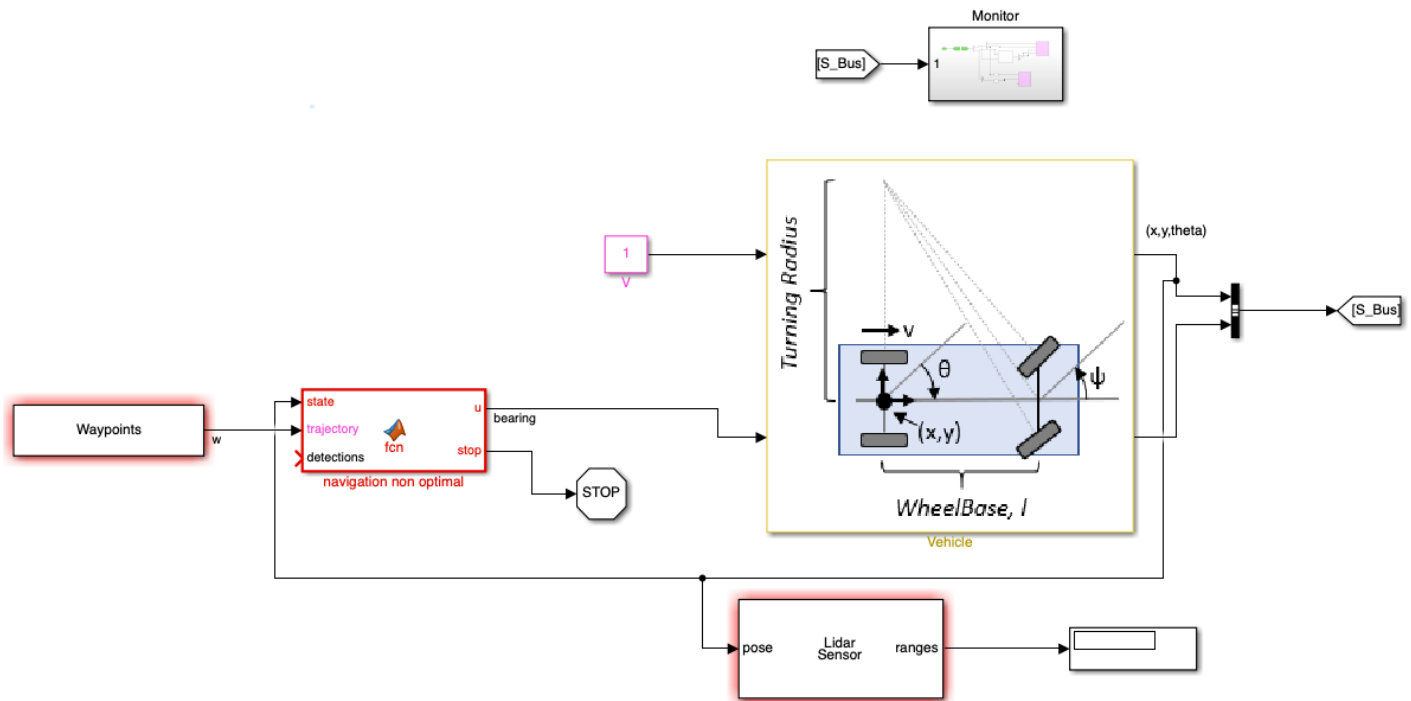
To add a Lidar and build an occupancy map of the environment, we can look into the library under **Mobile Robotic Simulation Toolbox->Environments**. See below.





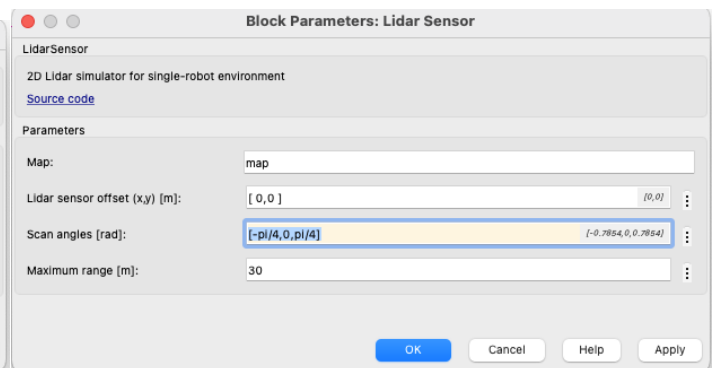
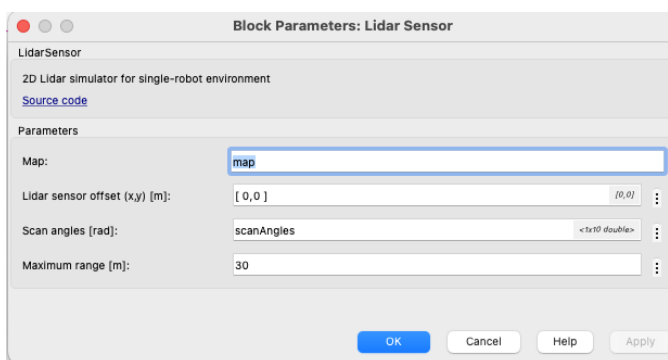


Once added to the model, it should look as below



double-click on the Lidar block to change parameters. Here you can modify the following:

- Scan angles: It is also known as angular resolution, i.e, how many scan lines per degree. It can be obtained from the workspace
- Maximum Range: Maximum Lidar range.
- Lidar offset: offset from the robot CoG.

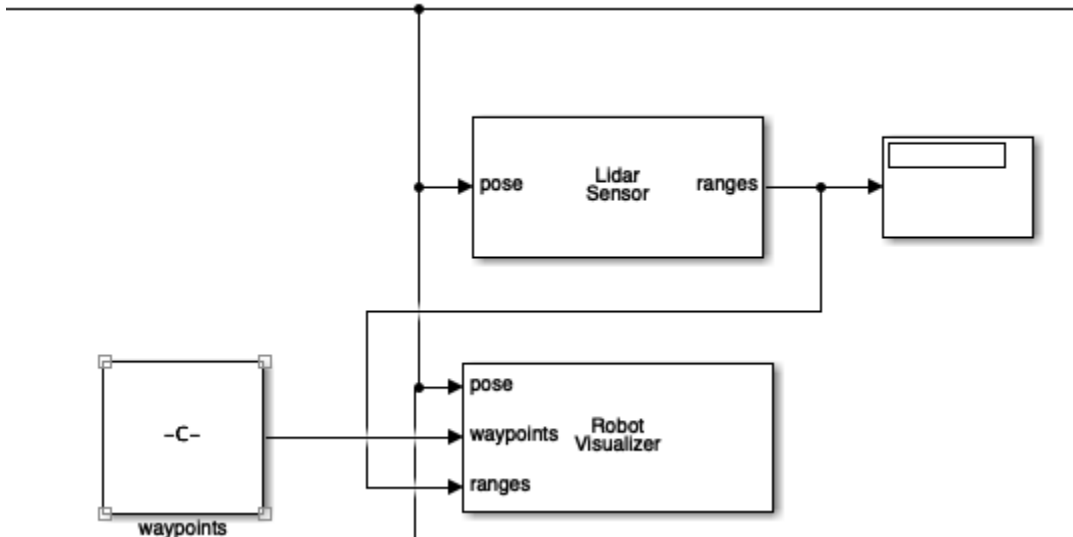


III. Disable Vehicle Viewer

Right-click on the block **Vehicle Viewer** and comment out the block. In this way, the visualiser (next) will display the map, waypoints and trajectory only.

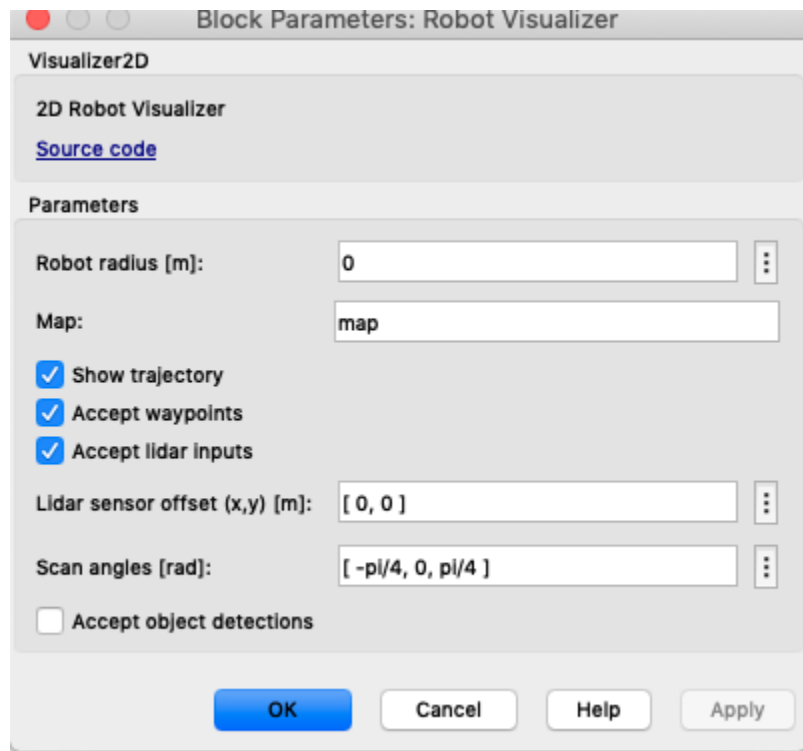
IV. Add robot and map visualisation

Add robot visualizer, from the library ([see above](#)).



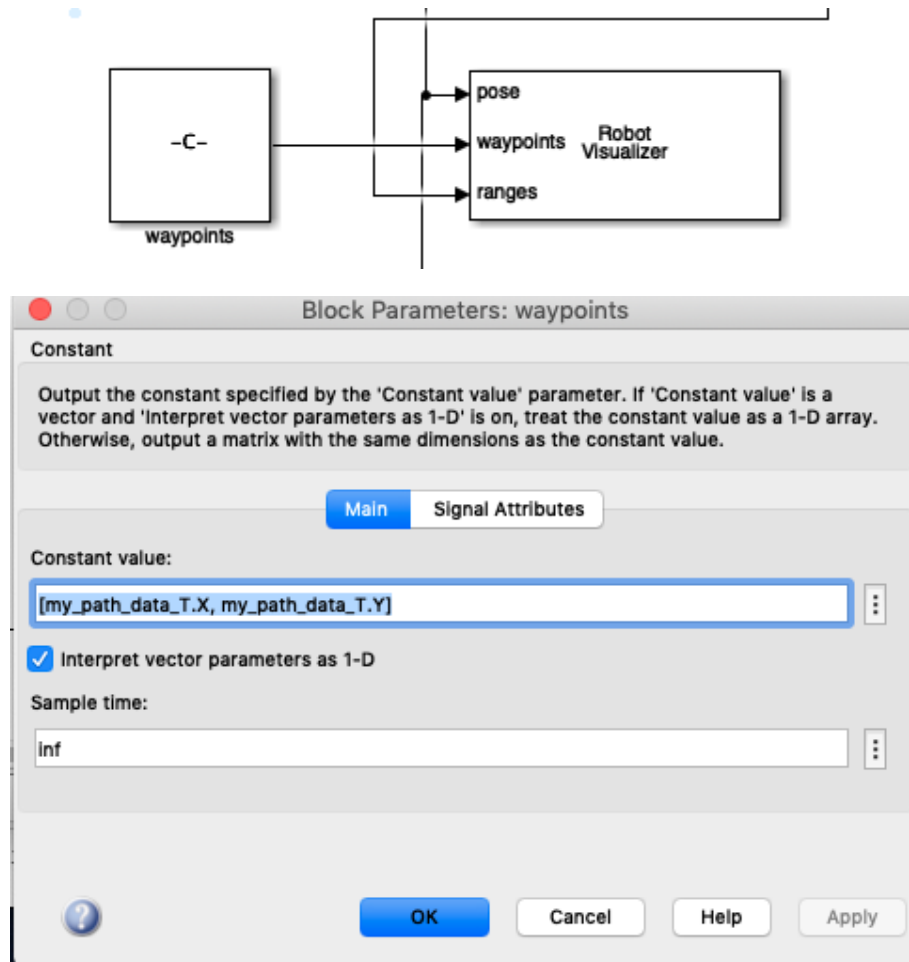
Double-click on the robot visualizer block, a window like one below should appear. This allows to configure

- The map (navigation area),
- Lidar parameters. They should match the parameters in the Lidar block
- In addition, you select to display waypoints, trajectory and accept Lidar ranges.



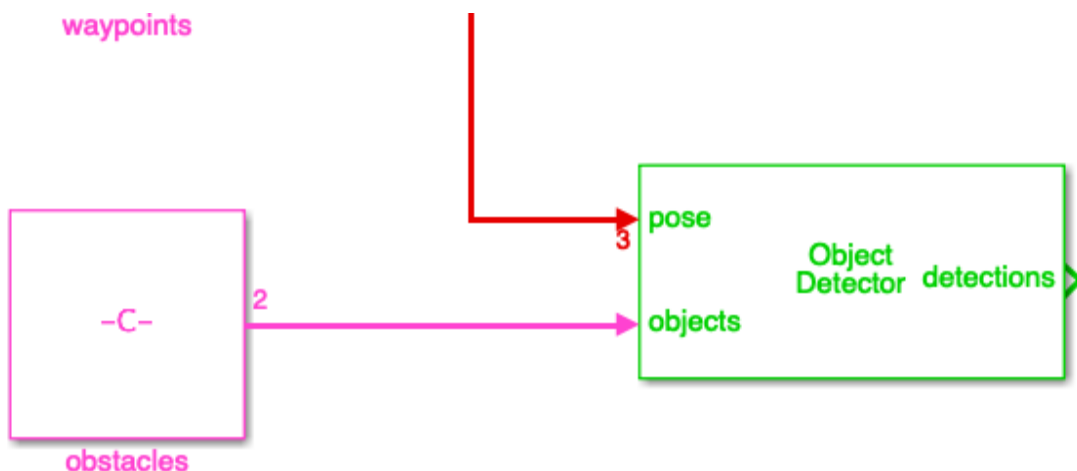
Try different parameters for Scan angles or angular resolution and Maximum Range.

As waypoint input, try adding a constant block with an array of waypoints as follows

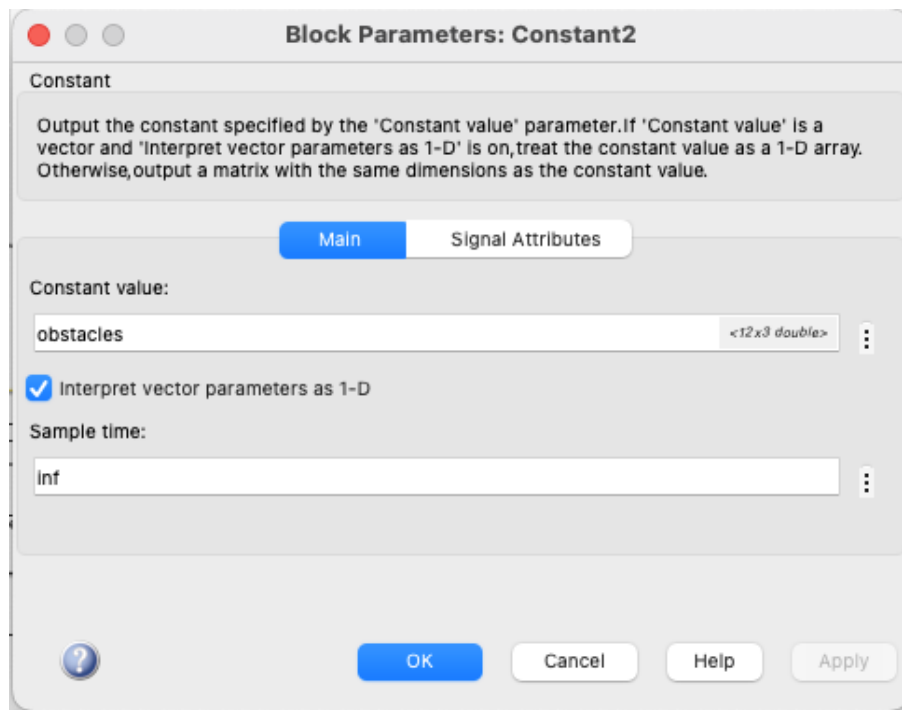


V. Add a map and obstacles.

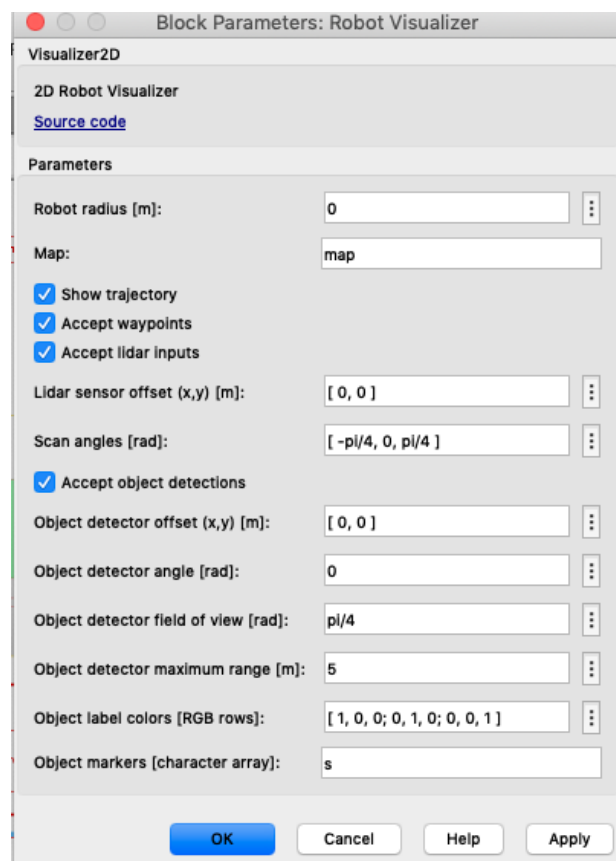
Once the map and obstacles are available in the workspace. We can now, add obstacles to the visualisation. From the library (see section II) select the object detector and add it to the environment. Obstacles can be added as a constant block. Double-click to edit on the constant block



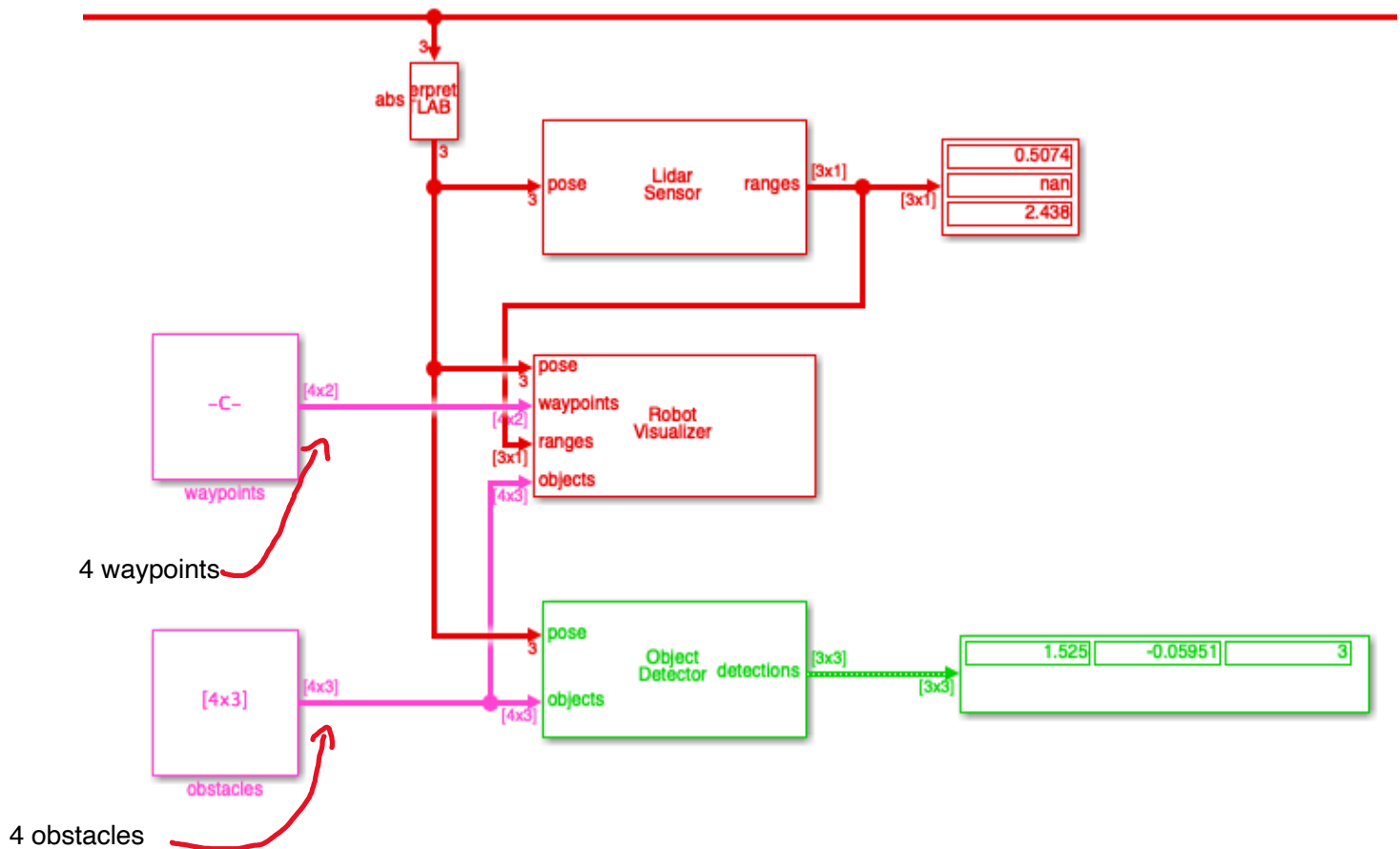
And add the name of the variable obstacles to it. (see below)



Once obstacles are added, double-click the **robot visualizer** block and check the box **accept object detections**, make sure the object detector field of view matches your lidar field of view.



Finally, connecting the blocks **object detector** and **robot visualizer** should enable the visualisation of obstacles. The system should look like.



VI. Adding collision avoidance logic

The final activity involves adding a logic so the vehicle uses the information output by the **object detector** block and **Lidar** block to avoid obstacles on the path. There are many options to implement this logic, here are some suggestions:

1. The block object detector outputs [range, angle, label], uses range to switch to a local reactive control to avoid obstacles and then returns to the waypoint guidance logic once the obstacle is avoided.
2. Define intermediate waypoints around the obstacles and insert these into the current guidance logic.
3. Using 1, find the gaps between obstacles and walls, find the relative angle of these gaps and command the vehicle through them.
4. Define an obstacle boundary (choose a circle or given radius), find a point on the circle and command the vehicle to that waypoint, once the obstacle is avoided return to high-level guidance (waypoint following).